CSD201 PE INSTRUCTIONS

Read the instructions below carefully before you begin coding.

Students may use only the following:

- Materials available on their computer (including JDK, NetBeans...).
- For distance learning: Google Meet, Hangout (for exam monitoring purposes).

Follow these steps to complete the PE:

1. Create a folder to store the given projects, e.g. CSD_given (1). Download the given materials into (1).
2. Implement the requirements of the test.
3. **Before submission:**
   - Clean and Build Project (Shift+F11)
   - Then rename the folder **dist** to **run**. (If a folder named run already exists, delete it before renaming).
4. **Submission:**
   - To submit project Q1, you must first select Question No: 1
   - Browse and select the project folder (e.g. Q1, or Q1A, or Q2, ...)
   - Then click the **Submit** button.
5. **Do not use accented Vietnamese** when writing comments in programs.
6. **Do not add** new **import** statement(s) to given files.
7. The software tools required are: **NetBeans IDE 8.x** and **Java JDK 1.8**. **If at least one of the above requirements is not followed, the exam will get ZERO.**

Notes:

- The input and expected output shown below are used **only to test your codes**.
- The input and expected output in the **real testcases** (for marking) are **different** from the examples below.
- **Do not hardcode** your code against the given expected results.

Questions: (10 marks)

The provided files already contain code skeletons for implementing a program that manages a **House moving process**. The structure of the main classes is described below:

- Class **Item**: stores information about a furniture item, including:
  - **name** (e.g., "Table", "Chair", "Sofa")
  - **weight** (must be > 0, representing the weight of the item in kg)
  - **length** (must be > 0, indicating the number of meters the item occupies)
  - **fragile** is 1 (if the Item is **fragile**), or 0 (if this is **not fragile**)
- Class **Node**: consists of **info** (an Item object) and **next** (a pointer for linking).
- Class **ItemList**: is a **singly linked list** that manages regular Nodes holding complete information about the furniture in the house.
- Class **Truck**: is a **stack** structure (implemented as a linked list) whose Nodes hold the furniture items loaded onto the truck. The stack has FIFO properties; in this test it is built using a singly linked list, with items **pushed** to the **head**.
- Class **House**: is the main class of the program, containing an ItemList used to manage the furniture and a Truck corresponding to the truck that transports the furniture.

Students must carefully read the provided code segments to fully understand the relationships between the classes and the functions within each class. The specific task of the test is to implement the following requirements:

a. f1(): 2.5 marks – Load data

To complete requirement f1(), students must:

1. Implement the function **addLast(String name, int weight, int length, int fragile)** in the **ItemList** class - list
2. Implement the function **push(String name, int weight, int length, int fragile)** in the **Truck** class - truck

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

The expected output used to test your code is shown below:

```
ItemList Inventory: (Stove, 10, 2, 0) (Lamp, 5, 1, 1) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (TV, 20, 1, 1)
(Chair, 7, 1, 0)
Truck Contents: (Pot, 10, 1, 1) (Bed, 100, 2, 0)
```

b. f2(): 2.5 marks – Load the first fragile item onto the truck

To complete requirement f2(), students must:

1. Implement the **deleteFirstFragile()** function to remove the **first fragile** item from the **list**.
2. Then **push** this item into the **truck**.

The expected output used to test your code is shown below:

```
ItemList Inventory: (Stove, 10, 2, 0) (Lamp, 5, 1, 1) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (TV, 20, 1, 1)
(Chair, 7, 1, 0)
Truck Contents: (Pot, 10, 1, 1) (Bed, 100, 2, 0)
ItemList Inventory: (Stove, 10, 2, 0) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (TV, 20, 1, 1) (Chair, 7, 1, 0)
Truck Contents: (Lamp, 5, 1, 1) (Pot, 10, 1, 1) (Bed, 100, 2, 0)
```

c. f3(): 2.5 marks – Load all fragile items onto the truck

To complete requirement f3(), students must:

1. Remove **all fragile items** from the list.
2. Then push each removed item into the truck in the same order they were removed from the original list.

The expected output used to test your code is shown below:

```
ItemList Inventory: (Stove, 10, 2, 0) (Lamp, 5, 1, 1) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (TV, 20, 1, 1)
(Chair, 7, 1, 0)
Truck Contents: (Pot, 10, 1, 1) (Bed, 100, 2, 0)
ItemList Inventory: (Stove, 10, 2, 0) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (Chair, 7, 1, 0)
Truck Contents: (TV, 20, 1, 1) (Lamp, 5, 1, 1) (Pot, 10, 1, 1) (Bed, 100, 2, 0)
```

d. f4(): 2.5 marks – Compute total weight of items on the truck

To complete requirement f4(), students must:

1. Remove **all fragile items** from the list. (as f3)
2. Push each removed item into the truck in the same order as they appeared in the original list. (as f3)
3. Compute and return the **total weight** of all items in the truck.

The expected output used to test your code is shown below:

```
ItemList Inventory: (Stove, 10, 2, 0) (Lamp, 5, 1, 1) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (TV, 20, 1, 1)
(Chair, 7, 1, 0)
Truck Contents: (Pot, 10, 1, 1) (Bed, 100, 2, 0)
ItemList Inventory: (Stove, 10, 2, 0) (Table, 80, 3, 0) (Sofa, 50, 2, 0) (Chair, 7, 1, 0)
Truck Contents: (TV, 20, 1, 1) (Lamp, 5, 1, 1) (Pot, 10, 1, 1) (Bed, 100, 2, 0)
Total weight: 135
```

===End of test===